Introductory Programming Techniques

North Metropolitan TAFE

ICTPRG302: Session 4

Courses:

- Certificate III in Information Technology
- Certificate IV in IT (Programming)
- Certificate IV in IT (Networking)
- Certificate IV in IT (Cybersecurity)
- Certificate IV in IT (Web)

What will I learn?

- The definition of a loop
- The two types of loops (while and for)
- Combining loops with if, elif and else statements
- Break and continue statements.

To complete this week

- Weekly session exercises
- Portfolio questions
- Revision…practise, practise, practise!

Revision: if statement

True or **False?**

The following code will print "Good morning!"

```
x = 10
if x == 10:
    print("Good morning!")
```

Revision: elif statement

True or False?

The elif statement can be used without an initial if statement.

```
time_of_day = 10

elif 12 <= time_of_day < 18:
    print("Good morning, how are you?")
else:
    print("Good afternoon, how are you?")
```

Revision: elif statement

Corrected code.

```
time_of_day = 10

if time_of_day < 12
      print("Good morning, how are you?")
elif 12 <= time_of_day <18
    print("Good afternoon, how are you?")
else:
    print("Good evening, how are you?")
```

Revision: else statement

True or False?

The elif statement ALWAYS follows an else statement.

```
x == 10

if x > 0:
    print("x is larger")
else:
    print("x is smaller")
elif:
    print(“x is equal”)
```

Revision: else statement

Corrected code.

```
x == 10

if x > 0:
    print("x is larger")
elif x == 0:
    print("x is equal")
else:
    print("x is smaller")
```

Reminder: Additional resources

Free online courses, to enhance your knowledge.

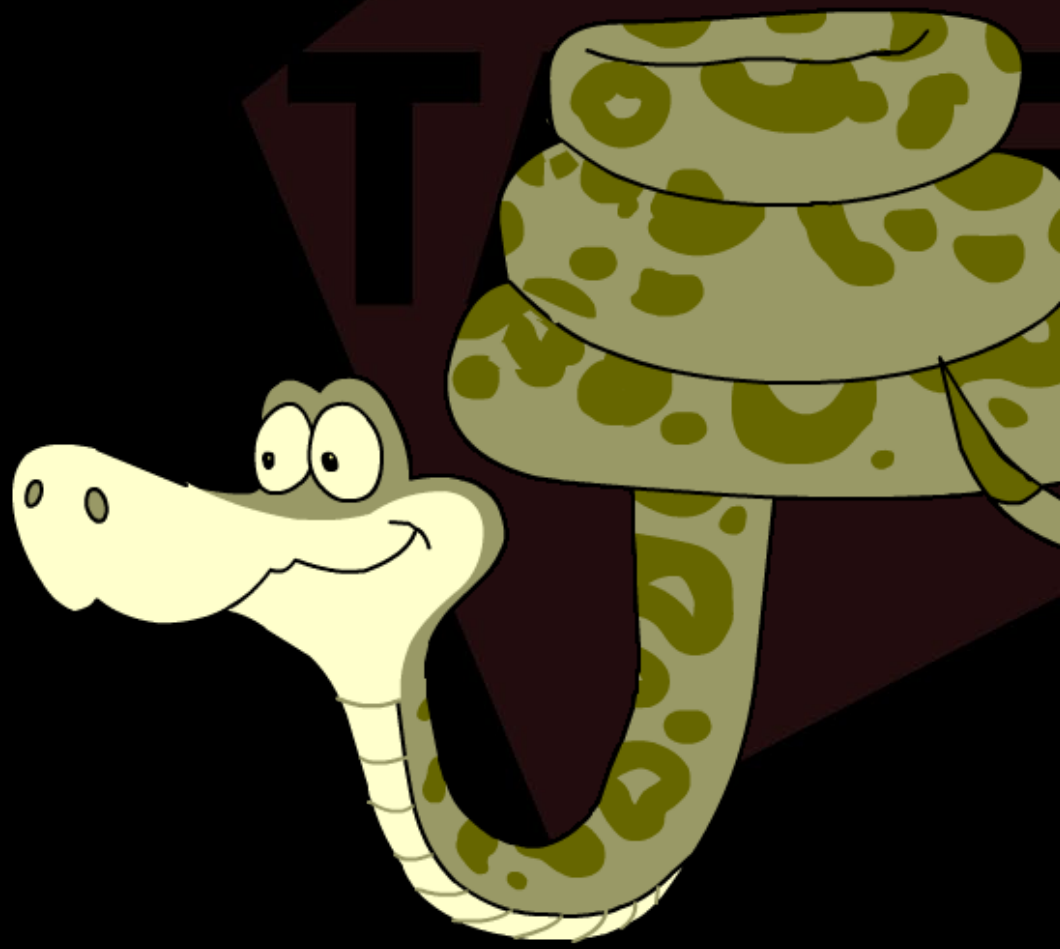
Python for Everyone (Py4e)

LearnPython.org

W3schools

What is a Loop?

A Python loop, allows you to repeat a block of code multiple times.



Iteration

A sequence of instructions that repeats until a specified condition is met is called an iteration.

Python Loops are useful because…

- You don’t have to write the same code multiple times
- Automates repetitive tasks, improves efficiency/time
- Improved code readability and maintainability.

Types of Loops

In Python, there are two types of loops.

A **while loop** and a **for loop**.

Why use a while loop?

A while loop repeats a block of code

while a certain condition is **True**.

When the condition is no longer True (False),

the loop terminates (ends).

Let's code!

Example: while loop

Syntax:

```
number = 0
while number < 5:
        print(number)
        number = number + 1
```

Output:

```
0
1
2
3
4
```

Example: while loop explained

Syntax:

```
number = 0
while number < 5:
    print(number)
    number = number + 1
```

Iteration steps:

Number is 0, which is less than 5.
Prints 0.
Increments number to 1 (0 + 1 = 1).

Number is 1, which is less than 5.
Prints 1.
Increments number to 2 (1 + 1 = 2).

Number is 2, which is less than 5.
Prints 2.
Increments number to 3.

And so on, until the number is 5.
The condition number < 5 is False, so the loop stops.

Why use a for loop?

- Useful when you need to repeat a block of code
- Executes a block of code for each item in the sequence
- Repeats a specific number of times over a specific sequence of items
- Less prone to creating infinite loops.

Example 1: for loop

prints each letter in “apple” (in CAPS).

Syntax:

```
for each_letter in "apple":
    print(each_letter.upper())
```

Output:

```
A
P
P
L
E
```

Example 1: for loop explained

Syntax:

```
for each_letter in "apple":
    print(each_letter.upper())
```

Iteration steps:

The for loop starts by setting the variable each_letter to iterate over each character in the string "apple".

The loop will go through each character in the string one by one. In this case, it will iterate over 'a', 'p', 'p', 'l', and 'e'.

Inside the loop, the print(each_letter.upper()) statement is executed. The upper() method converts the current character (each_letter) to its uppercase.

The print() function outputs the uppercase version of each character.

Example 2: for loop

prints number within range

Syntax:

```
for my_number in range(5):
    print(my_number)
```

Output:

```
0
1
2
3
4
```

***NOTE:** The numbers print from 0 because of how the range() function works in Python. By default, range(number) generates a sequence of numbers starting from 0 up to, but not including, number.
If you want to start from a different number, you can specify a starting point in the range() function.
For example: range(1, 6) would generate the sequence 1, 2, 3, 4, 5. Further explained in the upcoming Lists session.

Example 2: for loop explained

Syntax:

```
for my_number in range(5):
    print(my_number)
```

Iteration steps:

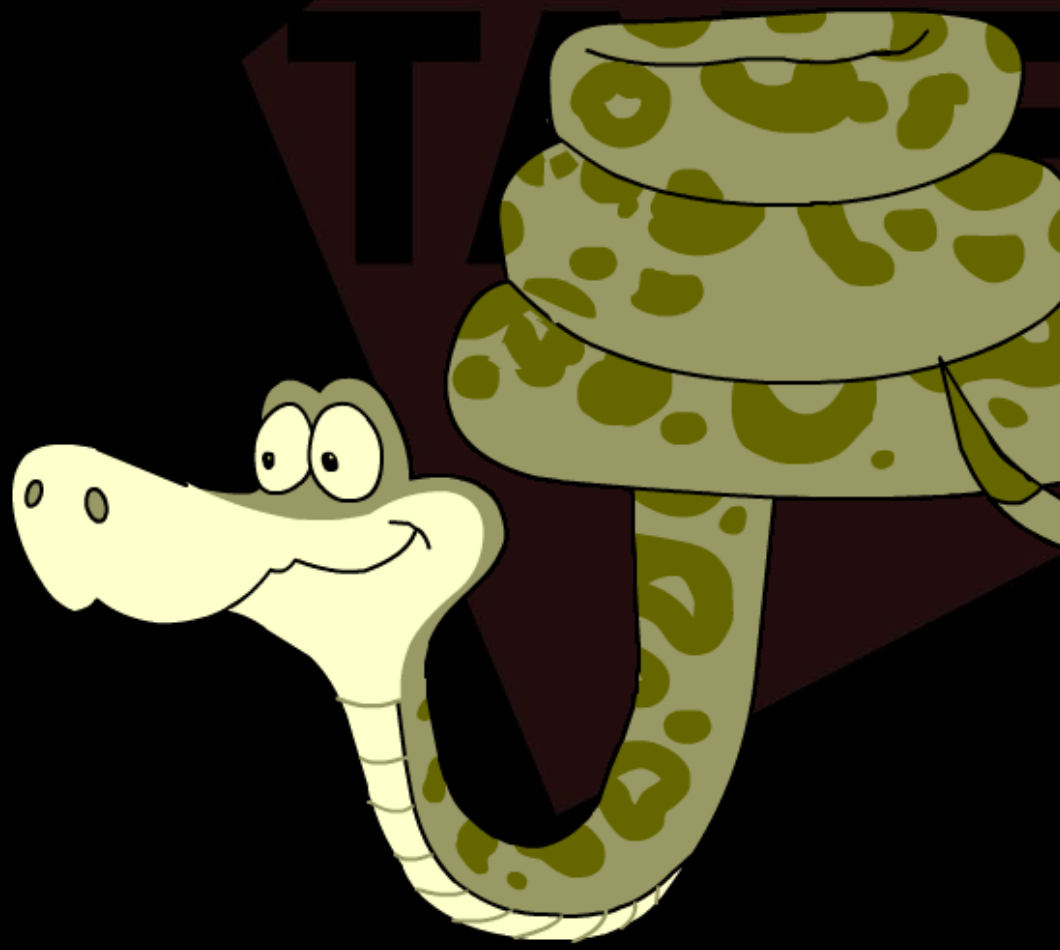
range(5): Generates a sequence of numbers from 0 to 4.

for my_number in range(5): iterates over each number in the sequence.

print(my_number): prints the current value of my_number.

Infinite Loops

An infinite loop will continue to execute if a terminating condition is never met.



Example: Infinite Loop

What is wrong with this while loop?

Syntax:

```
count_down = 1
while count_down > 0:
    print("Blast off!")
```

Example: Infinite Loop explained

Syntax:

```
count_down = 1
while count_down > 0:
    print("Blast off!")
```

Iteration steps:

count_down is set to 1 which is greater than 0.

The condition count_down > 0 is always True because count_down is never updated in the loop.

The loop continues to print "Blast off!" indefinitely.

Definite Loops

Since a *for* loop iterates (repeats) a specific number of times, it is categorised as a “definite” loop.

- The number of iterations is known before the loop starts.
- We say "definite loops iterate" through a sequence with a predetermined length.

Example: Definite Loop explained

This loop will run exactly 5 times, printing numbers 0 to 4.

Syntax:

```
for number in range (5):
    print(number)
```

Indefinite Loops

While loops are "indefinite loops" -- they keep going until a condition becomes **False**.

The number of iterations is unknown.

Example: Indefinite Loop explained

This loop will run until 'number' is no longer less than 5.

Syntax:

```
number = 0
while number < 5:
    print(number)
    number += 1
```

Breaking a loop: break statement

To break out of a loop, Python uses the break statement.

This statement immediately terminates the loop, regardless of the loop condition.

- Can be used in both for and while loops
- Used when you find the item you are looking for and no longer need to continue the loop.

Example: break explained

Syntax:

```
number = 0
while number < 10:
    if number == 5:
        break
    print(number)
    number += 1
```

Iteration steps:

The loop starts with number equal to 0.
It checks if number is less than 10.
Since 0 < 10 is True, it enters the loop.
It checks if number is equal to 5.
Since 0 == 5 is False, it skips the break statement.
It prints 0.
It increments number to 1.
The loop repeats, printing 1, 2, 3, and 4.
When number becomes 5, the condition number == 5 is True, so the break is executed, and the loop exits.

Continue statement

The continue statement skips the rest of the code inside a loop for the current iteration and moves to the next iteration.

It is often used to skip certain conditions without terminating the entire loop.

Example: Continue explained

Syntax

```
number = 0

while number < 10:
        if number == 5:
            number += 1
            continue
        print(number)
        number += 1
```

Iteration steps:

The loop starts with number equal to 0.
It checks if number is less than 10.
Since 0 < 10 is True, it enters the loop.
It checks if number is equal to 5.
If it is, the continue statement is executed, which skips the rest of the code inside the loop for that iteration and moves to the next iteration.
If number is not equal to 5, it prints the current value of number.
It increments number by 1.

More PEP8 Standards

Examples:

Correct	Incorrect
for number in range(5):	for number in range(5) :
print(number)	print(n)
number = 0	n = 0
while number < 5	while n < 5
number += 1	n+=1

Summary

- *while* loops (indefinite)
- *for* loops
- Using *break*
- Using *continue*
- More PEP8 standards

Activity to complete

Practise what you have learnt today!

Locate your Week 4 Revision exercises folder and complete the following activity:

1. Learn your Times Tables.